

DocIntel — Getting Started Guide

Overview

DocIntel is a document intelligence application for specialty insurance, built with Next.js and deployed on **Snowflake App Runtime**. You have read-only access to the source repository. To work with the code, fork it to your own GitHub account and deploy to your own Snowflake account.

Prerequisites

- **Git** (version 2.30+)
 - **Node.js** (version 20+)
 - **Snowflake CLI** (`snow` command) — [Install guide](#)
 - A **GitHub account** — request collaborator access from the repository owner
 - A **Snowflake account** with ACCOUNTADMIN access (for initial setup)
-

Step 1: Get Repository Access

1. Send your **GitHub username** to the repository owner
2. You'll receive an invitation email from GitHub
3. Accept the invitation at <https://github.com/notifications>

Once accepted, you'll have read-only access to view, clone, and fork the repository.

Step 2: Fork the Repository

1. Go to https://github.com/sfc-gh-tboon/document_extract_cowork
2. Click the **Fork** button (top right)
3. Select your GitHub account as the destination
4. Click **Create fork**

This creates your own copy at `https://github.com/YOUR-USERNAME/document_extract_cowork`.

Your fork is fully yours — you can edit, customize, and experiment freely. If something breaks, you can always revert to the original version by pulling from the source repository (see “Pulling Updates” below).

Step 3: Clone Your Fork

```
git clone https://github.com/YOUR-USERNAME/
    document_extract_cowork.git
cd document_extract_cowork
```

Step 4: Install the Snowflake CLI

macOS

```
brew install snowflake-cli
```

Windows / Linux

See: <https://docs.snowflake.com/developer-guide/snowflake-cli/installation/installation>

Verify

```
snow --version
```

Step 5: Configure Your Snowflake Connection

The Snowflake CLI uses a connections file to authenticate. Create or edit `~/.snowflake/connections.toml`:

```
[MY_CONNECTION]
account = "YOUR_ORG-YOUR_ACCOUNT"
user = "YOUR_USER"
authenticator = "externalbrowser"
```

```
role = "ACCOUNTADMIN"
warehouse = "COMPUTE_WH"
```

Replace `YOUR_ORG-YOUR_ACCOUNT` with your Snowflake account identifier (e.g., `MYORG-MYACCOUNT`) and `YOUR_USER` with your username.

Test it:

```
snow connection test --connection MY_CONNECTION
```

Step 6: One-Time Account Setup

Run the account provisioning script as ACCOUNTADMIN. This creates the `DOC_INTEL_ADMIN` role, compute pool, warehouse, and External Access Integration:

```
snow sql -f sql/00_provision_account.sql --connection
MY_CONNECTION
```

This only needs to be run once per Snowflake account. After this, all subsequent operations use the `DOC_INTEL_ADMIN` role.

Step 7: Configure deploy.config

Edit `deploy.config` in the project root to match your environment. The key settings to review:

Setting	Description	Default
<code>DOC_INTEL_DATABASE</code>	Database name	<code>DOC_INTEL</code>
<code>DOC_INTEL_QUERY_WH</code>	Warehouse for queries	<code>DOC_INTEL_QUERY_WH</code>
<code>DOC_INTEL_COMPUTE_POOL</code>	Compute pool for the app container	<code>DOC_MONITOR_POOL</code>
<code>DOC_INTEL_DEFAULT_USER</code>	Your Snowflake username	—

Setting	Description	Default
	(for role grants)	
<code>SNOWFLAKE_DEFAULT_CONNECTION_NAME</code>	Connection name from Step 5	—

Set `DOC_INTEL_DEFAULT_USER` to your Snowflake username so the deploy script grants you the appropriate roles.

Step 8: Deploy

The `deploy.sh` script handles everything in one command — database infrastructure, app deployment, and access grants:

```
./deploy.sh
```

What it does

Phase	Description
Phase 0	Pre-flight checks — verifies CLI, connection, and role
Phase 1	Database infrastructure — creates schemas, tables, stages, search services, agent, and RBAC
Phase 2	App deployment — runs <code>snow app deploy</code> to build and launch the web application
Phase 3	Post-deploy grants — configures caller's rights and role assignments
Phase 4	Verification — confirms all objects exist and the app is running

The first deploy takes 5-10 minutes. Subsequent deploys are faster (~3 minutes).

Deploy options

Flag	Effect
<code>--infra-only</code>	Database setup only, skip the app
<code>--app-only</code>	Redeploy the app only (infrastructure already exists)
<code>--skip-sample-data</code>	Don't load sample documents
<code>--dry-run</code>	Preview what would execute without running anything

Step 9: Access Your Application

Once deploy completes, it prints the app URL. You can also find it with:

```
snow app status
```

The URL looks like:

```
https://xxxxx-your-account.snowflakecomputing.app
```

Open it in your browser. Users authenticate with their Snowflake credentials automatically — no separate login system.

Tearing Down

To remove everything (database, app, roles) and start fresh:

```
./undeploy.sh
```

How Snowflake App Runtime Works

Snowflake App Runtime deploys web applications directly into Snowflake:

- **No Docker required** — Snowflake builds the container from your source code
- **No credentials to manage** — Authentication uses Snowflake's identity layer

- **Caller's rights** — SQL executes as the logged-in user, inheriting their RBAC permissions
- **Data stays in Snowflake** — No data egress, no external API layers
- **Enterprise SSO** — Users log in with existing Snowflake credentials (SSO, MFA supported)

For more: <https://docs.snowflake.com/en/developer-guide/snowflake-app-runtime/about-snowflake-app-runtime>

Project Structure

```

document_extract_cowork/
├─ deploy.sh           # Full deployment script (run this to deploy)
├─ undeploy.sh         # Tear down everything
├─ deploy.config       # Configuration (database, warehouse, roles, user)
├─ sql/               # SQL scripts executed by deploy.sh
│  └─ 00_provision_account.sql # One-time ACCOUNTADMIN setup
│  └─ 01_foundation.sql      # Database, schemas, roles
│  └─ 02_ingestion.sql       # Registry, stage, procedures
│  └─ 03_processing.sql      # Document catalog, extractions
│  └─ 04_telemetry.sql       # Events, metrics
│  └─ 05_intelligence.sql    # Semantic view, search, agent
│  └─ 06_admin.sql          # Config, queues, feedback
│  └─ ...
├─ app/               # Next.js application (deployed by deploy.sh)
│  └─ app/             # Pages and API routes
│  └─ components/      # React components
│  └─ lib/             # Snowflake client, auth, utilities
│  └─ snowflake.yml     # App Runtime project definition
│  └─ app.yml          # Build and run manifest
└─ docs/              # Documentation

```

Pulling Updates

To get the latest changes from the source repository into your fork:

```

# First time only — add the upstream remote
git remote add upstream https://github.com/sfc-gh-tboon/
                           document_extract_cowork.git

# Pull and merge latest changes
git fetch upstream
git merge upstream/main

```

To completely reset your fork to match the original (discards your changes):

```
git fetch upstream
git reset --hard upstream/main
git push --force
```

Troubleshooting

Cannot see the repository or “Fork” button

- Check that you’ve accepted the collaborator invitation at <https://github.com/notifications>
- Contact the repository owner with your GitHub username to be added

Deploy fails with “compute pool not found”

Either create the pool manually:

```
CREATE COMPUTE POOL DOC_MONITOR_POOL
MIN_NODES = 1
MAX_NODES = 1
INSTANCE_FAMILY = CPU_X64_S;
```

Or edit `deploy.config` to use an existing compute pool in your account.

“SNOWFLAKE_HOST not set” in the running app

The app runs inside Snowflake App Runtime — it cannot run standalone outside of Snowflake. Always deploy with `./deploy.sh` or `snow app deploy`.

Build errors during deploy

Check the build logs:

```
snow app logs
```

Common causes: missing Node.js 20+, or `package-lock.json` out of sync (run `npm install` locally then commit).

Need Help?

- `docs/DEMO_SCRIPT.md` — Walkthrough of all application features

- `sql/` — Database provisioning scripts with inline comments
- [Snowflake App Runtime docs](#)
- Contact the repository owner for access issues